
FONDAMENTI DI PROGRAMMAZIONE B

Tempo a disposizione: 2 ore

Nome Cognome Matricola

Esercizio 1 [C++] (15pt). Definire una classe templatica `Bag<T>` che realizza il tipo di dato astratto collezione (bag) di elementi di tipo `T`. Una bag ammette duplicati ma non mantiene ordine. La classe deve definire:

- ▶ un costruttore senza argomenti che crea una bag vuota
- ▶ un metodo `void add(T elem)` che aggiunge un elemento alla bag
- ▶ un metodo `int count(T elem)` che restituisce il numero di occorrenze di `elem` in una bag
- ▶ un metodo `contains(T elem)` che verifica se `elem` è presente almeno una volta
- ▶ un metodo `removeOne(T elem)` che rimuove solo una occorrenza di `elem`, se `elem` non è presente deve lanciare un'eccezione
- ▶ un metodo `int size()` che restituisce il numero totale di elementi nella bag

Si sovraccarichi l'operatore `+` in modo tale che restituisca una nuova bag contenente tutti gli elementi delle due bag (incluso duplicati). Si sovraccarichi l'operatore `<<` in modo tale che stampi gli elementi della bag su uno stream di output `fout` nel formato: $\{e_1, e_2, \dots, e_n\}$. Non è consentito utilizzare classi della STL. Se necessario, ridefinire gli opportuni metodi, costruttori e/o operatori. Specificare opportunamente eventuali metodi e parametri costanti. Massimizzare incapsulamento e information hiding.

Esercizio [Java] (15pt). Nel contesto dell'organizzazione dei Mondiali di calcio 2026, si implementino le seguenti classi.

- ▶ Si implementi una classe astratta `Persona` caratterizzata da nome (`String`) e cognome (`String`). La classe deve fornire un costruttore che inizializza i campi, fornire i metodi `getter`, ridefinire il metodo `equals` (due persone sono uguali se hanno stesso nome e cognome). Dichiarare un metodo astratto: `public int getValore();`
- ▶ Si implementino le classi `Giocatore` e `Allenatore`, sottoclassi di `Persona`. La classe `Giocatore` è caratterizzata da numero di gol segnati (`int`) e deve fornire un costruttore, ridefinire il metodo `equals` (stessi dati + stessi gol) e implementare `getValore` restituendo il numero di gol. La classe `Allenatore` è caratterizzata da anni di esperienza (`int`) e deve fornire un costruttore, ridefinire il metodo `equals` e implementare `getValore` restituendo gli anni di esperienza.
- ▶ Si implementi la classe `Squadra` caratterizzata da nome (`String`) e insieme di oggetti di tipo `Persona`. La classe deve fornire un costruttore senza parametri (squadra inizialmente vuota) e implementare il metodo: `addPersona(Persona p)` che aggiunge una persona alla squadra. Se la persona è già presente, il metodo deve lanciare un'eccezione non controllata `SquadraException` (da implementare). Implementare il metodo `int valoreTotale()` che ritorna la somma dei valori (usando `getValore`) di tutte le persone della squadra. Ridefinire il metodo `equals` (due squadre sono uguali se hanno stesso nome e stessi membri)
- ▶ Si implementi la classe `Mondiale` caratterizzata da un insieme di squadre. La classe deve avere un costruttore senza parametri e implementare il metodo: `addSquadra(Squadra s)` che aggiunge una squadra; se già presente deve lanciare `SquadraException`. Implementare il metodo: `getMigliore()` che restituisce la squadra con valore totale massimo.
- ▶ (+3pt) La classe `Squadra` deve implementare l'interfaccia `Comparable<Squadra>` utilizzando il valore totale per il confronto.

N.B. Massimizzare incapsulamento e information hiding. Non è richiesta l'implementazione del metodo `hashCode` per le classi richieste. Per ciascuna classe, è possibile supporre di avere a disposizione una implementazione del metodo `hashCode` coerente col metodo `equals` che implementerete.